

m-format constellation -- Command-line Examples

mnemonic-toolkit v0.55.3 -- worked examples (Linux), exact verbatim I/O

2026-06-15

Contents

About these examples	2
Seed input from a file	2
1. Install the constellation on Linux	3
2. Single-sig card set from a seed phrase (file input)	5
3. Conventional 2-of-3 multisig from 3 seed phrases (per-device, file input)	6
4. Card set -> Bitcoin Core wallet descriptor (and how to import)	8
5. Custom degrading-miniscript wallet (distinct keys per tier) + watch-only export	10
5.1 The secret word (a hashlock shared by tiers 1 and 2)	10
5.2 The guided builder caps complexity -- use the raw descriptor path	10
5.3 Engrave the card set	11
5.4 Watch-only export for Nunchuk / Core / Sparrow	15
6. Taproot version of the degrading wallet	16
6.1 Why depth-1 (the one-tier-per-leaf limit on master)	16
6.2 Build + validate	16
6.3 Engrave + first address	16
6.4 Export for wallets (Nunchuk / Core / Sparrow)	17
6.5 Adding a condition: which depth? (a spending-cost comparison)	18
Appendix -- the public test seeds used	20

About these examples

This document shows real, copy-pasteable command lines for the **m-format constellation** -- a steel-engravable Bitcoin backup system built around four CLIs (mnemonic, md, ms, mk). Every command below was executed against mnemonic **v0.55.3** on Linux and **both the command and its full output are reproduced verbatim** -- no abbreviations, no ellipses, no elided keys or addresses. Long lines wrap with a grey hook-arrow continuation marker in the left margin.

All seed phrases here are public BIP-39 TEST VECTORS (abandon abandon ... about, etc.). They are world-known and hold no funds. **Never type a real seed phrase onto a networked machine, and never reuse these test phrases for real money.** The toolkit is alpha software -- use only with disposable amounts or on testnet until it has been independently audited.

Seed input from a file

Yes -- the toolkit reads a seed phrase from a file. There is no `--phrase-file` flag; instead every secret slot accepts the value `-`, meaning **"read this secret from stdin"**, which you point at a file with the shell's `< file` redirect:

```
$ mnemonic bundle --template bip84 --network mainnet --slot @0.phrase=- < seed.txt
```

This is the **secure** idiom: the phrase travels on stdin, so it never appears in `argv` / `/proc/$PID/cmdline` or your shell history (unlike the inline form `--slot @0.phrase='<words>'`, which the toolkit flags as `secret-on-argv`). The reader strips a single trailing newline, so an ordinary one-line text file works. Only **one** stdin secret is allowed per invocation, so file-on-stdin fits any single-seed command; multi-seed wallets (section 3) gather one public xpub per device instead.

The three engraved cards:

Card	What it carries
ms1	BIP-39 entropy (recovers the seed)
mk1	xpub + origin (master fingerprint + BIP path)
md1	wallet policy (descriptor template + bound xpubs)

Throughout, `$` is the shell prompt; everything after it is what you type.

1. Install the constellation on Linux

The in-repo installer builds each component with `cargo install --locked` into `~/.cargo/bin` (no `sudo`, no system files touched). It needs `cargo`, `git`, and a C toolchain; the CLIs require `rustc >= 1.85`.

Install all four CLIs (this compiles from source, so the build log is machine-specific and not reproduced here):

```
$ sh -c "$(curl -fsSL https://raw.githubusercontent.com/bg002h/mnemonic-toolkit/master/scripts/install.sh)" -
- --no-gui
```

The installer carries the current version pins, so it never goes stale. Useful flags: `--only <c>`, `--exclude <c>`, `--no-gui`, `--from-git`, `--force`, `--dry-run`, `--list`. The pin table (`--list`) and a dry run are deterministic:

```
$ sh '/scratch/code/shibboleth/mnemonic-toolkit/scripts/install.sh' --list
```

COMPONENT	CARGO_PACKAGE	DEFAULT	FEATURES	GIT_TAG
-----	-----	-----	-----	-----
mnemonic	mnemonic-toolkit	git (only)	(none)	mnemonic-toolkit-v0.55.3
md	md-cli	crates.io	cli-compiler	descriptor-mnemonic-md-cli-v0.6.2
ms	ms-cli	crates.io	(none)	ms-cli-v0.7.0
mk	mk-cli	crates.io	(none)	mk-cli-v0.8.0
mnemonic-gui	mnemonic-gui	git (only)	(none)	mnemonic-gui-v0.40.0

```
$ sh '/scratch/code/shibboleth/mnemonic-toolkit/scripts/install.sh' --no-gui --dry-run
```

```
m-format constellation installer
```

```
install root: /home/bcg/.cargo/bin
```

```
source: crates.io (default; mnemonic-toolkit stays on git+tag)
```

```
install mnemonic (git: mnemonic-toolkit-v0.55.3)
```

```
[dry-run] cargo install --locked --git https://github.com/bg002h/mnemonic-toolkit --tag mnemonic-toolkit-v0.55.3 mnemonic-toolkit
```

```
install md (crates.io: md-cli)
```

```
[dry-run] cargo install --locked --features cli-compiler md-cli
```

```
install ms (crates.io: ms-cli)
```

```
[dry-run] cargo install --locked ms-cli
```

```
install mk (crates.io: mk-cli)
```

```
[dry-run] cargo install --locked mk-cli
```

```
skip mnemonic-gui
```

```
4 installed.
```

```
verify:
```

```
mnemonic --version      md --version
```

```
ms --version            mk --version
```

```
mnemonic-gui --version
```

Verify the install and list every subcommand:

```
$ mnemonic --version
```

```
mnemonic 0.55.3
```

```
$ mnemonic --help
```

```
engraving-bundle CLI for the m-format star (ms1 + mk1 + md1)
```

Usage: `mnemonic [OPTIONS] <COMMAND>`

Commands:

`bundle` emit a 3-card engraving bundle from a phrase or xpub

`verify-bundle` round-trip-check an engraved bundle

`convert` convert between seed/key formats (BIP-39 / BIP-32 / WIF / ms1 / mk1)

`addresses` list a wallet's receive/change addresses (batch, read-only)

`decode-address` decode a Bitcoin address → network(s) / script type / witness version / scriptPubKey

`export-wallet` emit watch-only wallet artifacts (Bitcoin Core `importdescriptors`, BIP-388 `wallet_policy`)

import-wallet	import a third-party wallet blob into an m-format bundle (v0.26.0 Phase 2: BSMS Round-2 only)
derive-child	derive deterministic child entropy / keys from a master xprv (BIP-85)
electrum-decrypt	decrypt an Electrum field-encrypted secret (seed phrase / xprv) with a password
final-word	emit the set of BIP-39 last words that yield a valid checksum for an N-1 partial phrase
seed-xor	split a BIP-39 phrase into N XOR shares OR combine N shares back into a phrase
seedqr	encode/decode SeedQR (BIP-39 mnemonic ↔ numeric digit-string QR payload)
nostr	Wrap an existing nostr key (npub/nsec) as Bitcoin addresses/descriptors/WIF
silent-payment	Derive a BIP-352 silent-payment receiver address (base + labeled) from a seed
slip39	split a master secret into SLIP-39 K-of-N shares OR combine shares back (Trezor-compatible)
ms-shares	split a secret into BIP-93 codex32 K-of-N (msl) shares OR combine shares back
gui-schema	emit SPEC §7 GUI-overlay flag-surface schema JSON (companion to `mnemonic-gui` v0.2)
repair	BCH error-correct a corrupted m-format card (msl / mk1 / mdl)
inspect	describe the contents of an m-format card (msl / mk1 / mdl)
compare-cost	compare wsh-vs-tr per-spending-condition cost for a miniscript or descriptor
xpub-search	search for a target (xpub, descriptor, address, or passphrase) under a seed or xpub
verify-message	verify a Bitcoin message signature (legacy P2PKH signmessage + BIP-322 segwit/taproot)
restore	emit a watch-only restore document (single-sig) from a seed + optional passphrase
build-descriptor	build a validated wsh(...) descriptor + BIP-388 policy from a JSON policy-tree spec
help	Print this message or the help of the given subcommand(s)

Options:

--no-auto-repair v0.22.0 – skip auto-fire repair on decode failures; preserve pre-v0.22 exit policy. Global flag. Honor bundle`. For `verify-bundle`, auto-fire is additionally gated on `std::io::stdout().is_terminal()` to preserve the legacy V row behavior when output is piped or captured (per v0.22.1 D18 – TTY-conditional default). Standalone `repair` ignores this f -json` calling contexts the auto-fire emits a structured JSON envelope on stdout (per v0.22.1 D20) instead of text-form

-h, --help	Print help
-V, --version	Print version

RECOVERING A FORGOTTEN BIP-39 PASSPHRASE:

If you have your seed words (entropy) but not the BIP-39 passphrase (the optional "25th word"): if you have a LIST of likely passphrases, `mnemonic xpub-search passphrase-of-xpub --passphrase-candidates-file <file> --target-xpub <a known xpub>` tests each candidate against a value you already know. To GENERATE or mutate a keyspace (wordlists, masks, typo models), `mnemonic` does not – an external open-source tool does: btcrecover searches passphrase candidates and confirms each by deriving an address / xpub / master-fingerprint at common default paths and matching a value you already know.

btcrecover (maintained): <https://github.com/3rdIteration/btcrecover>
original: <https://github.com/gurnec/btcrecover>

Pointer current as of 2026-05-25. Run untrusted recovery tools offline, on an air-gapped machine.

2. Single-sig card set from a seed phrase (file input)

Create a native-segwit (BIP-84, `m/84'/0'/0'`) single-sig 3-card bundle from one seed phrase held in a file. Write the phrase to `seed0.txt` (here a public test vector) and feed it on stdin:

```
$ printf '%s\n' 'abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about' > seed0.txt
$ cat seed0.txt
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about

(--template choices for single-sig: bip44, bip49, bip84, bip86.) Run the bundle. stdout carries the three cards to
engrave; stderr carries the human-readable engraving panel and the secret-material warning:

$ mnemonic bundle --template bip84 --network mainnet --slot @0.phrase=- < seed0.txt
# msl (entropy, BCH-checksummed)
msl0entrsqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq9sraq34v7f

msl0e ntrsqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq9 sraq 34v7f

# mkl (xpub + origin)
mkl1qprsqhpqqsq3cqtstleutks2qvz3vs70mejhk622ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0z8v4yslsp8qp3yxg3dpe854wq4
mkl1qprsqhppo0f30mtxzd65mvwcur9usdatwuqvq6z70r9nwrqk6xn6l8gy6nwa2n977sw6zh34rma0nh

mkl1qp rsqhp qqsq3 cqtstl eeutk s2qvz g3vs7 0mejh k622w s2kgd
emj2c d8zwj 2skzx 2wq0q w70l4 q99vd yh5x0 z8v4y slsp8 qp3yx
g3dpe 854wq 4
mkl1qp rsqhp p0f30 mtxzd 65mvw cur9u sdatw uqvq6 z70r9 nwrqk
6xn6l 8gy6n wa2n9 77sw6 zh34r ma0nh

# mdl (wallet policy)
mdl1fgdxlppqpm6jzzqqvqpdqw0za5zs4ggy55aq4vsmnh4s6wyaypu34c7raqu8np
mdl1fgdxlppqf2zcgefcupmel75q5435j7seugaj5jr7qyur6vt76es5cdeyrq7zdy0d
mdl1fgdxlppq3xa2dk8vwpj7gx74hwqxqdp083jehp5tdrfa0n5zdfkqcdlrvnh5r62jn

mdl1fg-dxlpq-pqpm6-jzzqq-vqpdq-w0za5-zs4gy-y55aq-4vsmn-hy4s6-wyayp-u34c7-raqu8-np
mdl1fg-dxlpq-f2zcg-efcpu-pmel7-5q543-5j7se-ugaj5-jr7qy-ur6vt-76es5-cdeyr-q7zdy-0d
mdl1fg-dxlpq-3xa2d-k8vwp-j7gx7-4hwqx-qdp08-3jehp-5tdrf-a0n5z-dfkqc-dlrvn-h5r62-jn

# === Wallet bundle: bip84, mainnet ===
# msl: 1c017
# mkl: 1c017
# fingerprint: 73c5da0a
# origin path: m/84'/0'/0'
# Template: bip84
# mdl: 1c01
warning: stdout carries private key material (can spend) – redirect or encrypt (e.g. '> file.txt' or '| age -
e ...')
```

Each card is printed twice: once unbroken, once grouped into 5-character blocks (`msl0e ntrsqqqqq ...`) -- the grouped form is what you punch or engrave. Add `--no-engraving-card` to suppress the stderr panel when piping into other tools.

3. Conventional 2-of-3 multisig from 3 seed phrases (per-device, file input)

A real multisig never lets one machine see more than one seed. Each cosigner derives **only their public xpub** from their own seed file (on their own, ideally air-gapped, device); the coordinator then combines the three **public** keys into a watch-only `wsh(sortedmulti(...))`. No secret ever leaves its file.

Put each cosigner's seed in its own file:

```
$ printf '%s\n' 'abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about' > seed0.txt
$ printf '%s\n' 'legal winner thank year wave sausage worth useful legal winner thank yellow' > seed1.txt
$ printf '%s\n' 'letter advice cage absurd amount doctor acoustic avoid letter advice cage above' > seed2.txt
```

On each device, derive that cosigner's BIP-87 multisig fingerprint and account xpub (`--template wsh-sortedmulti` implies the `m/87'/0'/0'` path) from the seed file:

```
$ mnemonic convert --from phrase=- --to fingerprint --template wsh-sortedmulti --network mainnet < seed0.txt
fingerprint: 73c5da0a

$ mnemonic convert --from phrase=- --to xpub --template wsh-sortedmulti --network mainnet < seed0.txt
xpub: xpub6DBjYnc4ewKti13Q1L35bqdodw5z3VGJnf516B3icHrEGEUcCuCG5GVQDZtH8Xmsyt3Fs9YDNwLaqjUbbRidwXZ6sxufZcr4VqqzrXvicM
note: stdout is watch-only – public keys only, cannot spend

$ mnemonic convert --from phrase=- --to fingerprint --template wsh-sortedmulti --network mainnet < seed1.txt
fingerprint: b8688df1

$ mnemonic convert --from phrase=- --to xpub --template wsh-sortedmulti --network mainnet < seed1.txt
xpub: xpub6CbhrPzY2z7NcCGCgJLAJLq8iRyjUfwmDXQs66MxTVURkQb9DpLnVJ5D1qpatZjUuPGTyxf5TYU1vA34YFE9FHB4TvfYmokYLVsyEFZFt9
note: stdout is watch-only – public keys only, cannot spend

$ mnemonic convert --from phrase=- --to fingerprint --template wsh-sortedmulti --network mainnet < seed2.txt
fingerprint: 28645006

$ mnemonic convert --from phrase=- --to xpub --template wsh-sortedmulti --network mainnet < seed2.txt
xpub: xpub6DB7HNqW6CZojxN85NuFTPWZhi2FagSnexPS1rv3nYQHngkmdHgb7iebYvTFmFKKDA3ozf5yezDsCH6cXAw3WZijviSZtZC2hjHn2uazz4z
note: stdout is watch-only – public keys only, cannot spend
```

Wrap each as an origin-annotated descriptor key `[fingerprint/87'/0'/0']xpub` and combine into a 2-of-3 sorted-multisig descriptor (`/<0;1>/*` = the external/change multipath). The assembled descriptor file:

```
$ cat multisig.desc
wsh(sortedmulti(2,[73c5da0a/87'/0'/0']xpub6DBjYnc4ewKti13Q1L35bqdodw5z3VGJnf516B3icHrEGEUcCuCG5GVQDZtH8Xmsyt3Fs9YDNwLa
```

Validate and canonicalise it (this also computes the BIP-380 checksum):

```
$ mnemonic export-wallet --descriptor "$(cat multisig.desc)" --format descriptor --network mainnet
wsh(sortedmulti(2,[73c5da0a/87'/0'/0']xpub6DBjYnc4ewKti13Q1L35bqdodw5z3VGJnf516B3icHrEGEUcCuCG5GVQDZtH8Xmsyt3Fs9YDNwLa
note: stdout is watch-only – public keys only, cannot spend
```

The first receive address (here via the BSMS / BIP-129 record, which also carries the `/0/*,/1/*` derivation):

```
$ mnemonic export-wallet --descriptor "$(cat multisig.desc)" --format bsms --network mainnet
BSMS 1.0
wsh(sortedmulti(2,[73c5da0a/87'/0'/0']xpub6DBjYnc4ewKti13Q1L35bqdodw5z3VGJnf516B3icHrEGEUcCuCG5GVQDZtH8Xmsyt3Fs9YDNwLa
/0/*,/1/*
bclqkssenl2m6t3aynza394sr9m86vt6md2v76kj52jun2xlrdeaa4q84qtpl
note: stdout is watch-only – public keys only, cannot spend
```

Engrave the shared watch-only card set from the public descriptor (the `md1` policy card is shared by all cosigners; each cosigner additionally backs up their own seed as a single-sig `ms1` set per section 2). With only public xpubs supplied, the `ms1` cards are empty placeholders:

```
$ mnemonic bundle --descriptor-file multisig.desc --network mainnet
# ms1 (omitted – descriptor watch-only mode)
```

```

# mk1[0] (cosigner 0 xpub + origin)
mk1qperpupqqspu3s7denyv8nwverpumnrnchdq5pcy3zepa59349dcgs5n5stvpvk3v8x4eqsdngy2jl4wddt5ac2ptv4fya76tpreapyrdqfqr
mk1qperpuppnq8kp6xcpphr7svxwx5ag99s9zfym19v7tcqrexmpdj7jgqgmny8rr0z7vlj7eqzv2486xkkcftrzq8

mk1qp erpup qqspu 3s7de nyv8n wverp umnrn chdq5 pcy3z epa59
349dc gs5n5 stvpv k3v8x 4eqsd ngy2j l4wdd t5ac2 ptv4f ya76t
preap yrdfq r
mk1qp erpup pnq8k p6xcp qphr7 svxwk xx5ag 99s9z fym19 v7tcq
rexmp dj7jg qgmny 8rr0z 7vlj7 eqzv2 486xk kcftr zq8

# mk1[1] (cosigner 1 xpub + origin)
mk1qperpapqqspu3s7denyv8nwverpum9cdzxlzpcy3zepaqxafil630m4as45q6fz4ltsntlues3e3gylcu6jsa6jdz69hy0whcpg2f28j5awu
mk1qperpapp02p3qtffjqpxtuj95jzaevqs7jqje40324vxlwf0txswawpxte3zmhzp6lrpj3ga2lw65h2dd24cuerkpa

mk1qp erpap qqspu 3s7de nyv8n wverp umn9c dzxlz pcy3z epa59
afl63 0m4as 45q6f z4lts ntlue s3e3g ylcu6 jsa6j dz69h y0whc
pg2f2 8j5aw u
mk1qp erpap p02p3 qtffj qpxtu j95jz aevqs 7jqje 40324 vxlfw
0txsw awpxt e3zmh zp6lr pj3ga 2lw65 h2dd2 4cuer kpa

# mk1[2] (cosigner 2 xpub + origin)
mk1qperp7pqspu3s7denyv8nwverpumnpvg3gqvpcy3zeganecw97sn9g25uukrgxca47at8as54xhd0rkgx57cs78mc8fk6507cs8tnugnpnf
mk1qperp7ppr4e59ef7u5p038j900ye439hsphz506fra5k3eac5kg0n7jhn3kpzpn6n0lygd1pfeq4a75lc2mrrz7f

mk1qp erp7p qqspu 3s7de nyv8n wverp umnpg v3gqv pcy3z epa59
cw97s n9g25 uukrg xca47 at8as 54xhd 0rkgx 57cs7 8mc8f k6507
cs8tn ugnpn f
mk1qp erp7p pr4e5 9ef7u 5p038 j900y e439h sphez 506fr a5k3e
ac5kg 0n7jh n3kpz pna6n 0lygd lpfeq 4a75l c2mrr z7f

# md1 (multisig wallet policy)
md1f5przzspq3m67zzqzvrs3pstucw0za5znwrg3hcc5xg5qxzhs7yyg2f6g9kqktgkrn2spp6tlfzprv6ye
md1f5przzswgyrv6pz5hatnt2a8wzs2m92ffsrmqarvqsqm3lgxr8trr2w5zjcz3yjd1jk0qf2g9tn3u5jtsz
md1f5przzsj7qq7fkctvh5jqzuepcmcchn8u30m4as45q6fz4ltsntlues3e3gylcu6jsasrwqtqpe4mvjxf
md1f5przzsafx3dzmj02p3qtffjqpxtuj95jzaevqs7jqje40324vxlwf0txswawpxte3zmhu35hx7lrg2t6
md1f5przz3r3qa03segkpx2s4feevxsd3mta6k0mpf2dw678vsdfa3pu0hswnvwhxsh98mjsqpsz8xkj9l6g
md1f5przz3gzl0y277fntzt0qr0j9gl5j8mfdm3fvs18a908rvzyr8m4xl7gs9e2t6dyvjgv4j

md1f5-przzs-pq3m6-7zzqz-vzrs3-pstuc-w0za5-znwrg-3hcc5-xg5qx-zhs7y-yg2f6-g9kqk-tgkrn-2spp6-tlfzp-rv6ye
md1f5-przzs-wgyrv-6pz5h-atnt2-a8wzs-2m92f-fsrmq-arvqs-qm3lg-xr8tr-r2w5z-jcz3y-jd1jk-0qf2g-9tn3u-5jtsz
md1f5-przzs-j7qq7-fkctv-h5jqz-xuepc-cmchn-8u30m-4as45-q6fz4-ltsnt-lues3-e3gyl-cu6js-asrwq-tqpe4-mvjxf
md1f5-przzs-afx3d-zmj02-p3qtf-fjqpx-tuj95-jzaev-qs7jq-je403-24vxl-fw0tx-swawp-xte3z-mspu3-5hx7l-rg2t6
md1f5-przz3-r3qa0-3segk-px2s4-feevx-sd3mt-a6k0m-pf2dw-678vs-dfa3p-u0hsw-nvwhx-sh98m-jsqps-z8xkj-h9l6g
md1f5-przz3-gzl0y-277f-ntzt0-qr0j9-gl5j8-mfdm3-fvs18a90-8rvzy-r8m4x-l7gs9-e2t6d-yvjgv-4j

# === Wallet bundle: descriptor, mainnet ===
# Threshold: 2 of 3
# Cosigners:
#   @0: (no ms1; watch-only),mk1:c8c3c (73c5da0a @ m/87'/0'/0')
#   @1: (no ms1; watch-only),mk1:c8c3d (b8688df1 @ m/87'/0'/0')
#   @2: (no ms1; watch-only),mk1:c8c3e (28645006 @ m/87'/0'/0')
# Template: descriptor
# md1: c8c3
# Recovery: any 2 of 3 signing keys + md1 (template card).
note: stdout is watch-only – public keys only, cannot spend

```

4. Card set -> Bitcoin Core wallet descriptor (and how to import)

`mnemonic restore --md1 <chunks>` reconstructs the watch-only wallet from the **shared md1 card alone** -- no seeds needed. First produce that card from the section-3 wallet and pull out its md1 chunks:

```
$ mnemonic bundle --descriptor-file multisig.desc --network mainnet --json > multisig.bundle.json
```

note: stdout is watch-only – public keys only, cannot spend

```
$ jq -r ".md1[]" multisig.bundle.json > multisig.md1
```

```
$ cat multisig.md1
```

```
md1f5przzspq3m67zzqqvzrs3pstucw0za5znwrg3hcc5xg5qxzhs7yyg2f6g9kqktgkrn2spp6tlfzprv6ye
md1f5przzswgyrv6pz5hatnt2a8wzs2m92ffsrmqarvqsqm3lgxr8trr2w5zjcz3yjd1jk0qf2g9tn3u5jtsz
md1f5przzsj7q7fkctvh5jqzxuepccmchn8u30m4as45q6fz4ltsntlues3e3gylcu6jsasrwqtqpe4mvjxf
md1f5przzsafx3dzmj02p3qtffjqpxtuj95jzaevqs7jqje40324vxlw0txswawpxte3zmspu35hx7lrg2t6
md1f5przz3r3qa03segkpx2s4feevxsd3mta6k0mpf2dw678vsdfa3pu0hswnvwhxsh98mjsqpsz8xkjh9l6g
md1f5przz3gzlz0y277fntzt0qr0j9gl5j8mfdrrnm3fvsl8a908rvzyr8m4xl7gs9e2t6dyvjgv4j
```

`Restore` reconstructs the wallet from exactly those chunks. The default form prints the descriptor and first address (note the address matches section 3 -- same wallet -- while the descriptor *string* differs because the md1 card stores each key as a depth-0 master xpub, an equivalent serialisation):

```
$ mnemonic restore --network mainnet $(sed "s/^/--md1 /" multisig.md1)
```

```
2-of-3 multisig restore
```

CONFIRM: verify each cosigner fingerprint against your records before importing.

```
descriptor: wsh(sortedmulti(2,[73c5da0a/87'/0'/0']xpub661MyMwAqRbcFrooZ2966EcDmVX5MoFXZhuJqXTudvJzwBTBfPQSc5JzX52fvS18o
```

```
first recv: bclqkssenl2m6t3aynza394sr9m86vt6md2v76kj52jun2xlwrdeaa4q84qtpl
```

```
cosigner @0: 73c5da0a [87'/0'/0'] from md1 (not independently verified)
```

```
cosigner @1: b8688df1 [87'/0'/0'] from md1 (not independently verified)
```

```
cosigner @2: 28645006 [87'/0'/0'] from md1 (not independently verified)
```

UNVERIFIED: no --from/--cosigner cross-check supplied; verify each cosigner fingerprint above against your records before importing

note: stdout is watch-only – public keys only, cannot spend

Add `--format bitcoin-core` for a ready-to-import `importdescriptors` request array (external `.../0/*` + change `.../1/*`):

```
$ mnemonic restore --network mainnet $(sed "s/^/--md1 /" multisig.md1) --format bitcoin-core
```

```
[
  {
    "active": true,
    "desc": "wsh(sortedmulti(2,[73c5da0a/87'/0'/0']xpub661MyMwAqRbcFrooZ2966EcDmVX5MoFXZhuJqXTudvJzwBTBfPQSc5JzX52fvS18o",
    "internal": false,
    "range": [
      0,
      999
    ],
    "timestamp": 0
  },
  {
    "active": true,
    "desc": "wsh(sortedmulti(2,[73c5da0a/87'/0'/0']xpub661MyMwAqRbcFrooZ2966EcDmVX5MoFXZhuJqXTudvJzwBTBfPQSc5JzX52fvS18o",
    "internal": true,
    "range": [
      0,
      999
    ],
    "timestamp": 0
  }
]
```

```
2-of-3 multisig restore
```

CONFIRM: verify each cosigner fingerprint against your records before importing the payload above.


```

descriptor: wsh(sortedmulti(2,[73c5da0a/87'/0'/0']xpub661MyMwAqRbcFrooZ2966EcDmVX5MoFXZhuJqXTudvJzwBTBfPQSc5JzX52fvS18
first recv: bclqkssenl2m6t3aynza394sr9m86vt6md2v76kj52jun2xlwrdeaa4q84qtpl
cosigner @0: 73c5da0a [87'/0'/0'] from md1 (not independently verified)
cosigner @1: b8688df1 [87'/0'/0'] from md1 (not independently verified)
cosigner @2: 28645006 [87'/0'/0'] from md1 (not independently verified)
UNVERIFIED: no --from/--cosigner cross-check supplied; verify each cosigner fingerprint above against your records before i
note: stdout is watch-only – public keys only, cannot spend

```

Import into Bitcoin Core: save the array, create a blank descriptor wallet, and load it (these run against your own node, so their output is not shown here):

```

$ mnemonic restore --network mainnet $(sed "s/^/--md1 /" multisig.md1) --format bitcoin-core > wallet.json
$ bitcoin-cli -named createwallet wallet_name="multisig-2of3" disable_private_keys=true blank=true descriptors=true
$ bitcoin-cli -rpcwallet="multisig-2of3" importdescriptors "$(cat wallet.json)"
$ bitcoin-cli -rpcwallet="multisig-2of3" getnewaddress

```

Tips: --timestamp now skips the rescan for a fresh wallet (default 0 rescans from genesis); --range 0,4999 widens the gap limit; --bitcoin-core-version 24 targets older Core. restore also emits --format descriptor (the bare wsh(...)#checksum) for other wallets.

5. Custom degrading-miniscript wallet (distinct keys per tier) + watch-only export

A four-tier vault. **Each tier uses its own distinct key set (no key reuse)**, deliberately mixing all four Bitcoin timelock kinds:

Tier	Spend condition	Timelock kind
1	3-of-3 (K0,K1,K2) + secret word	absolute height -- <code>after(1000000)</code>
2	2-of-3 (K3,K4,K5) + secret word	absolute time -- <code>after(1893456000)</code>
3	both K6 and K7	relative blocks -- <code>older(65535)</code>
4	any 1 of K8,K9,K10	relative time -- <code>older(4255898)</code>

That is **11 distinct keys** (3+3+2+3). Absolute locks (`after`) count from the chain's height/clock; relative locks (`older`) count from each coin's own confirmation. Encodings:

- `after(1000000)` -- absolute **block height** 1,000,000 (BIP-65; values below 500,000,000 are heights).
- `after(1893456000)` -- absolute **Unix time** = 2030-01-01 00:00 UTC (values at/above 500,000,000 are timestamps).
- `older(65535)` -- relative **blocks**: 65,535 blocks (~455 days). This is the largest safe relative-block lock; `older(65536)` would be BIP-68 consensus-masked to zero, and the toolkit warns if it sees one.
- `older(4255898)` -- relative **time**: BIP-68's time flag (bit 22) set, plus 61,594 units x 512 s $\approx$ 365 days.

5.1 The secret word (a hashlock shared by tiers 1 and 2)

Reusing a *hash* across tiers is fine -- it is not a key. The secret word is `opensesame`; the descriptor commits to `H = sha256(sha256(word))` and a spend reveals the 32-byte preimage `X = sha256(word)`:

```
$ python3 -c "import hashlib; w=b'opensesame'; X=hashlib.sha256(w).digest(); print('preimage X = ', X.hex()); print('hash H = ', hashlib.sha256(X).hexdigest())"
preimage X = cd6f70a4440de96063f8dbd7a4a3bbcff6b993af6c83e08d87b359e0760ca9c6
hash H      = a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad
```

5.2 The guided builder caps complexity -- use the raw descriptor path

`mnemonic build-descriptor` runs a satisfiability + cost preview that it **bounds** for funds-safety. An 11-key, 4-branch policy exceeds that envelope, so the guided builder refuses and points you at the raw `--descriptor` path:

```
$ mnemonic build-descriptor --spec policy.json --network mainnet
build-descriptor: refused - 1 diagnostic(s):
[over_envelope] root: policy exceeds the always-previewable envelope (2^(11 keys + 2 hashes) × 9 timelock-states > cap 4096); use the raw `--descriptor` path for arbitrarily complex policies
```

(For a policy *within* the envelope -- fewer keys -- `build-descriptor --spec` validates and emits it for you.) For arbitrarily complex policies you hand the miniscript descriptor straight to `export-wallet / bundle`. Validate and canonicalise it first (this adds the BIP-380 checksum). The full descriptor, with every xpub in full:

```
$ mnemonic export-wallet --descriptor "$(cat policy.desc)" --format descriptor --network mainnet
wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(1,1)),1)),1)/*
note: stdout is watch-only - public keys only, cannot spend
```

First receive address (Mainnet), via the BSMS record:

```
$ mnemonic export-wallet --descriptor "$(cat policy.desc)" --format bsms --network mainnet
BSMS 1.0
wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(1,1)),1)),1)/*
bclq4g7564xxd9hj68hqw5e558cqafhsklerkr0asfzqp6puq74veesrp6qss
note: stdout is watch-only - public keys only, cannot spend
```

5.3 Engrave the card set

Because every key is **distinct**, this is a valid BIP-388 wallet policy, so -- unlike a key-reusing policy -- `bundle` will engrave it. With only public xpubs supplied, the result is watch-only (the `ms1` cards are empty placeholders):

```
$ mnemonic bundle --descriptor-file policy.desc --network mainnet
# ms1 (omitted – descriptor watch-only mode)
```

```
# mkl[0] (cosigner 0 xpub + origin)
mklqptfcrzqqs945upmkpd8qwastfcrhvv6wqamqkns8wc95upmkpd8qwastfcrhvv6wqamqkns8wc95upmkpeutks2qvzgs7hd8pnnmcv56c4u
mklqptfcrzpkg08auetmd998g9tyxuae9vxn38f9gtp98q8s808l6szjkxjt6r83rk2jg0cqns0f30mtxxd65mvwcur9usv8qhcmu69gql0zrc
mklqptfcrzz74hwqxqdp083jehp5tdrfa0n5zdfhw6425sqe8pgsyxftjt4
```

```
mklqp tfcrz qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkp eutks 2qvzg s7hd8
pnnmc v56c4 u
mklqp tfcrz pkg08 auetm d998g 9tyxu ae9vx n38f9 gtp9 8q8s8
08l6s zjkxj t6r83 rk2jg 0cqns 0f30m txzd6 5mvwc ur9us v8qhc
mu69g ql0zr c
mklqp tfcrz z74hw qxqdp 083je hp5td rfa0n 5zdfh w6425 sqe8p
gsyxf tjtf4
```

```
# mkl[1] (cosigner 1 xpub + origin)
mklqptfczzqqs945upmkpd8qwastfcrhvv6wqamqkns8wc95upmkpd8qwastfcrhvv6wqamqkns8wc95upmkpeutks2lcpag7x7w00uvz59ukc0
mklqptfczzpszzqgqyzsqqgzyzsqgqzy3zepulhn90du6setz7wcggw0mnt0jarage0ptr0vxf3c9895ghkha0gk58rcwwcdclrvxvprnta
mklqptfczzdq82uxnprxcuejaentsxshgr9vudfrx5cc7npaegptf64nsud0nnnt334xdu0p83dn95g54tz
```

```
mklqp tfczz qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkp eutks 2lcpa g7x7w
00uvz 59ukc 0
mklqp tfczz pszqg qzyqs zqgqz ypsqz gqzqy 3zepu lhn90 du6se
tz7wc ggw0m nt0ja rage0 ptr0v xf3c9 895gh kha0g k58rc wwcdclrvxvprnta
mklqp tfczz zdq8 2uxnp rxcue jaent sxshg r9vud frx5c c7npa
egptf 64nsu d0nnn t334x du0p8 3dn95 g54tz
```

```
# mkl[2] (cosigner 2 xpub + origin)
mklqptfcpzqqs945upmkpd8qwastfcrhvv6wqamqkns8wc95upmkpd8qwastfcrhvv6wqamqkns8wc95upmkpeutks2lcpag07c7z8k5qayd8z5
mklqptfcpzpszzqgqyzsqqgzyzsqgqzy3zepulhn90d76enfqqwtug4zdpn8r94nrhgnwycc6l2errqlch68l94cdsr0vxvngvzwq0fhe86j
mklqptfcpzzqupqc34flc5lnh7dzkjmzwhfhtn2tf9tp8385pwa2rh2p9a6flzxmhl43hxx3cmgcpj625l8l
```

```
mklqp tfcpz qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkp eutks 2lcpa g07c7
z8k5q ayd8z 5
mklqp tfcpz pszqg qzyqs zqgqz yzsyz gqzqy 3zepu lhn90 d76en
fqwwt ug4zd pn8r9 4nrhg nwycc 6l2er rqlch 68l94 cdsr0 vxvngvzwq0fhe86j
mklqp tfcpz zqupq c34fl c5lnh z7dzk jmwzf htn2t f9tp8 385pw
pa2rh 2p9a6 flzxm hl43h hx3cm gcpj6 25l8l
```

```
# mkl[3] (cosigner 3 xpub + origin)
mklqptfcqzqqs945upmkpd8qwastfcrhvv6wqamqkns8wc95upmkpd8qwastfcrhvv6wqamqkns8wc95upmkpeutks2lcpagqk6f7csue6uzgna
mklqptfcqzpszzqgqyzsqqgzyzsqgqzy3zepulhn90d6awkdsm10jm7xjylhyyp7esaklwkwkasq24gnq3578mne3sjr64gdauj68uhy768d
mklqptfcqzzzqp0dmvslcy68sh0lw93nwwxth488eqjvn67gqjpx7fyf84xj7zjfc29xjtagpghsxz5xlu0rx
```

```
mklqp tfcqz qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkp eutks 2lcpa gqk6f
7csue 6uzgn a
```

mk1qp tfcqz pszqg qzyqs zqgqz yrszq gqzqy 3zepu lhn90 d6awk
dsm10 jm7xj ylhyy p7esa klkw kasq2 4gnq3 578mn e3sjr 64gda
uj68u hy768 d
mk1qp tfcqz zzqp0 dmvs1 cy68s h0lw9 3nwwx th488 eqjvn 67gqj
px7fy f84xj 7zjfc 29xjt agpgh sxz5x lu0rx

mk1[4] (cosigner 4 xpub + origin)

mk1qptfc8zqq945upmkpd8qwastfcrhvv6wqamqkns8wc95upmkpd8qwastfcrhvv6wqamqkns8wc95upmkzux3r03qvzgsnwq0e2f7mmwlnnx
mk1qptfc8zpk0w55t7uh3mhpjgm2ctzuevmwdfap2f5zl9lkdthf372tg8y9k3d35qm006q30450k4vmqvakpywv7pkdajnnrd36ymdaqsnv
mk1qptfc8z36y9wa86gpqpq5yyp22y3ww23fkwgjcckxsga8sgz6tl

mk1qp tfc8z qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkz ux3r0 3qvzg snwq0
e2f7m mwlnn x
mk1qp tfc8z pkg0w 55t7u h3mhp gjm2c tzuev mwdfp ap2f5 zl9lk
dthf3 72tg8 y9k3d 35qm0 06q30 450k4 vmqva kpywv 7pkda jnnrd
36ymd aqksn v
mk1qp tfc8z zm36y 9wa86 gpqpq 5yyp y22y3 ww23f xkwgj cskxq
sga8s gz6tl

mk1[5] (cosigner 5 xpub + origin)

mk1qptfcx8zqq945upmkpd8qwastfcrhvv6wqamqkns8wc95upmkpd8qwastfcrhvv6wqamqkns8wc95upmkzux3r03lcpagnlnxhkxfprlea74
mk1qptfcx8zpsqgqzyqsqgqzysqgqzy3zepa6j30mjj26l5afq22n2ct2h3zwrjpsefyfqr5zrzs70jdmqvx7wwt0z6a3fmdmqr0khu23
mk1qptfcx8zddqp6ttmcl57ac6vels0u3jj86dd5pe7m355rtt8r3a2s45tszcmrx9xa5qyec4g3crut8d54

mk1qp tfcxz qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkz ux3r0 3lcpa gnlnx
hkxfp rlea7 4
mk1qp tfcxz pszqg qzyqs zqgqz ypsqz gqzqy 3zepa 6j30m jj26l
5afq2 2n2ct 2h3zw rjpse fyfqr 5zrzs 70jdm tqvx7 wwt0z 6a3fm
fdmqr 0khu2 3
mk1qp tfcxz zdqp6 ttmc1 y57ac 6vels 0u3jj 86dd5 pe7m3 55rtt
w8r3a 2s45t szcmr x9xa5 qyec4 g3cru t8d54

mk1[6] (cosigner 6 xpub + origin)

mk1qptfc9zqq945upmkpd8qwastfcrhvv6wqamqkns8wc95upmkpd8qwastfcrhvv6wqamqkns8wc95upmkzux3r03lcpagz84k67v3r27gvvy
mk1qptfc9zpsqgqzyqsqgqzyzsqgqzy3zepa6j30mjemst4u6j4atq6uhd0m3m5l9lvu9cde6pcswdr73ejss4lts9qugnufy3yg6r2y0f
mk1qptfc9zvgpdy4ej7rexgzc7vsch5dxs9e587e8u22q38ujtrt54lm26dq30kfqqvm7wyvmtzy38suff

mk1qp tfc9z qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkz ux3r0 3lcpa gz84k
67v3r 27gvvy w
mk1qp tfc9z pszqg qzyqs zqgqz yzsqz gqzqy 3zepa 6j30m jemst
4u6j4 atq6u hd0m3 m5l9l vu9cd e6pcs wdr7 3ejss 4lts9 qugnu
fy3yg 6r2y0 f
mk1qp tfc9z zvqpq dye4j 7rexg zc7vs ch5dx s9e58 7e8u2 2q38u
jtrt5 4lm26 dq30k fqqvm 7wyvmtzy3 8suff

mk1[7] (cosigner 7 xpub + origin)

mk1qptfcy9zqq945upmkpd8qwastfcrhvv6wqamqkns8wc95upmkpd8qwastfcrhvv6wqamqkns8wc95upmkzux3r03lcpagd0hpxp2e6dx8r48
mk1qptfcy9zpsqgqzyqsqgqzyzsqgqzy3zepa6j30mjestc3wnact9lauqxytd7f9xhqmtmlma4zhsvtx5p6e3s3rggp2ce7mxam45j76f5qd
mk1qptfcy9z3qpzlzuww4vzvcksequdf17d8u34vrr7t6kyjwlagkep2ywk6g2q5h3z3tv9m06y2yd6kfgs

mk1qp tfcyz qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkz ux3r0 3lcpa gd0hp
xp2e6 dx8r4 8
mk1qp tfcyz pszqg qzyqs zqgqz yrszq gqzqy 3zepa 6j30m jestc3

wnact 9lauq xytd7 f9xhq mtm1m a4zhs vtx5p 6e3s3 rggp2 ce7mx
am45j 76f5q d
mk1qp tfcyz z3qz lzuvw w4vzv ckseq udf17 d8u34 vrr7t 6kyjw
lagke p2ywk 6g2q5 h3z3t vy9m0 6y2yd 6kfgs

mk1[8] (cosigner 8 xpub + origin)

mk1qptfctzqqs945upmkpd8qwastfcrhvx6wqamqkns8wc95upmkpd8qwastfcrhvx6wqamqkns8wc95upmkq5xg5qxqvzgs9n3l96x69pda2ys
mk1qptfctzpkg0dqcwzpsyrk9a808d5f0gennj3x9vjds63rqqgzndx78e533rulqyzd58cmvq4nudnky9jhqsh5zpemaskg7pdgxcftukrgv6n5
mk1qptfctzzt4c4w6rtnjmnht889ltvhne3ktx5encnny6spl8gsqf53tn98

mk1qp tfctz qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkq 5xg5q xqvzg s9n3l
96x69 pda2y s
mk1qp tfctz pkg0d qcwzp syrk9 a808d 5f0ge nnj3x 9vjds 63rqq
zndx7 8e533 rulqy zd58c mvq4n udnky 9jhqs h5zpe maskg 7pdgx
cftuk rgv6n 5
mk1qp tfctz zt4c4 w6rtn jmnht 889lt vhne3 ktx5e ncnnny 6spl8
gsqf5 3tn98

mk1[9] (cosigner 9 xpub + origin)

mk1qptfctzqqs945upmkpd8qwastfcrhvx6wqamqkns8wc95upmkpd8qwastfcrhvx6wqamqkns8wc95upmkq5xg5qxlcpg9zzktxfdleumyfr
mk1qptfctzpszqqgzyqsqqgzpszqqgzqy3zepa5rpcgx0jlag0vc85ztty3las62hfw73f3j8zx6t3lmtpe8skfujv0xfu3hslfcdrrrp8me0
mk1qptfctz2zlcpr80c5zmmvxxqknngp3d279fuywf7zfw5kqe8wv93v9fvn0m39u6adlnvgwullstspxm44n

mk1qp tfc2z qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkq 5xg5q xlcpa g9zzk
txfdl eumyfr
mk1qp tfc2z pszqq qzyqs zqqgz ypszq qqzqy 3zepa 5rpcg x0jla
g0vc8 5ztty 3las6 2hfw7 3f3j8 zx6t3 lmtpe 8skfu jv0xf u3hsl
fcdrr rp8me 0
mk1qp tfc2z zlcpr 80c5z mmvxx qknng p3d27 9fuyw f7zfw 5kqe8
wv93v 9fvn0 m39u6 5adln vgwul lstsp xm44n

mk1[10] (cosigner 10 xpub + origin)

mk1qptfctfzqqs945upmkpd8qwastfcrhvx6wqamqkns8wc95upmkpd8qwastfcrhvx6wqamqkns8wc95upmkq5xg5qxlcpg56yxxwr4asa24nc
mk1qptfctfzpszqqgzyqsqqgzyszqqgzqy3zepa5rpcgxwd86v5gunengsxd2x3nhwsl8dfjs79jwhy7qnx8r24vls4gl05j6uxcc7dl2rjmec
mk1qptfctfz2ypsarcmrl7tgutcg8uf4m7gfc8l fv33knprwthl06am5gdw9aqnfx4jz8l2kjr0p0ur43e8j

mk1qp tfcfz qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkq 5xg5q xlcpa g56yx
xwr4a sa24n c
mk1qp tfcfz pszqq qzyqs zqqgz yzszz qqzqy 3zepa 5rpcg xwd86
v5gun engsx d2x3n hws18 dfjs7 9jwhy 7qnx8 r24vl s4gl0 5j6ux
cc7dl 2rjme c
mk1qp tfcfz z2yps arcmr l7tgu tcg8u f4m7g fcd8l fv33k nprwt
hl06a m5gdw 9aqnf x4jz8 l2kjr 0p0ur 43e8j

md1 (multisig wallet policy)

md1ffumxts9z3m6jzqaafpr802gfgaafp9nh4yypm6jzxw75sj3m6jzt802ggrh4yyvaafp9genj3xrjhltxq
md1ffumxtsgqpxp2v6mq85yszv6a4pxuusy2unu8xqxz8naa2r2akwukvgm42gws7xkx9k7ctqmfdfe9crr4q
md1ffumxtssmngy26xzqqyn9xddhpk7cspxdw6snwvgzt4wf7rnqpre774p4wm8wtxyd64yxex7jxulxvqvkc
md1ffumxtsapudvvtas2de5z9drq3rg4jnxhqqqrl17xppvaxdwqqs8sngvq9zdqeccpeutk5lpf37mlhphsr
md1ffumxt3zpgtnchdq5feutks2xu79mg9yhp5gmu2ms6yd794cdzxlz7ux3r03s2ry2qrf9qs784qr5aw8s98
md1ffumxt3wg5qx52ry2qrpdkssz22ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0zxwq65s8c69ev9z
md1ffumxt34j5jr7qyur6vt76esnw4xmkr8qe0yr02mhqrxshncevms69k357he6px5mrn2zz033uqkqk8lql
md1ffumxt3u4308vyy88ae4hew375vhs43hkrycuznj6ytm7h5t2r3u8dqp82uxnprxcuejudws6tu9ntlfdx
md1ffumxtj8xdwq6zaqv4n34yv6nrr6v8h9q9d82kwr347wwd9mtxdypee03z5f5xvuvkkvwu4ep1qpedw7ye5

md1ffumxtjvfhzvvd04v33s0utarlj6uxcpkqupqc34flc5lnhz7dzkjmzwfhtn2tf9tp83xfskawee093tj
md1ffumxtjks9c84gwagyhfh8ugmw8t46ekr0a7t0c6gn7uss8mxrkma6e6mkqp24zvzxcnlwk74rja82tmur
md1ffumxtjeuccfpazqp0dmvslcy68sh0lw93nwwxth488eqjvn67gqjpx7fyf84xj7zjfc2wr604yuru2eq34
md1ffumxtnqams5fd4v93wvkdhx557s4y6p0jlmx45mcl995rjzmgkc6qdhhhaqgh668m2kdsxy5ru7p2fk5qgs
md1ffumxtnmfvzg ueurvmm8w8gs4m5lfqyqyzssyy3fgj9ee29zj26l5afq22n2c2t2h3zwrj46trnchtrjptz
md1ffumxtnsxr9y3yqwsqv2re7fhdvpsmeeeduttgqw6778e98hwxn0urlyv537ntdqw0kusp3l3wyry2c2
md1ffumxtne55rtw8r3a2s45tszcmrrfmst4u6j4atq6uhd0m3m5l9lvu9cde6pcswdr73cph9wc0utcf6l5
md1ffumxt5x2zzhawq5rzqgrfxdvhs7fjqk8nyx9arf5pdpkflzjsyflyjc6a9076kngytumwtunf5hft8gc
md1ffumxt5wtstc3wnact9lauqxytd7f9xhqmtmlma4zhsvtx5p6e3s3rggp2ezqz979cuua2mfvauyk5ut07p
md1ffumxt5nqn95xg8r20lnflydtqclj743ynhl29kg23r4kjz99qyrk9a808d5f0gennjslsrznrr484e6
md1ffumxt5uc4jfk2yvpq2d5mclxjxy0nuqsfsklsrdsk03kwcsk2uzz7sg880kzer6aw9tk9ypxcwzcyw6fy
md1ffumxt4zrtnjmnh889ltvhn3ktx5e8uhl2rmxpaqj6ey0lvxj46th52vv3c3kju0767gdjm2nna3x5pw
md1ffumxt4f8skfujv0xfalsrwxl3g9hkccvdpd8xsr264u2ncgunuyjafvpjwuctz2cjexlhznktnnzzu3sml7
md1ffumxt430x4tnf7n9z8y7v6ypn235vam58em2v583vn4e8sy3c64t8u928ma9zqcw3udszulgfsxu8xeyl
md1ffumxt4lluk3chss0cnthusns607jerrdxzxuh07l4mhgs6ut6pxjq3gs642g78tg45

md1ff-umxts-9z3m6-jzqaa-fpr80-2gfga-afp9n-h4yyp-m6jzx-w75sj-3m6jz-t802g-grh4y-yvaaf-p9gen-j3xrxj-hltjx-q
md1ff-umxts-gqpx-2v6mq-q85ys-zv6a4-pxuus-yh2un-u8xqz-8naa2-r2akw-ukvgm-42gws-7xkx9-k7ctq-mfdfe-9crr4-q
md1ff-umxts-s5mng-y26xz-zqyn9-xddhp-k7csp-xdw6s-nwggz-t4wf7-rnqpr-e774p-4wm8w-txyd6-4yxex-7jxuL-xvqvk-c
md1ff-umxts-apudv-vtdas-2de5z-9drq3-rg4jn-xhqqq-rll7x-ppvax-dwqqs-8sngv-q9zdz-eccpe-utk5l-pf37m-lhphs-r
md1ff-umxt3-zpgtn-chdq5-feutk-s2xu7-9mg9y-hp5gm-u2ms6-yd794-cdzxl-z7ux3-r03s2-ry2qr-f9qs7-84qr5-aw8s9-8
md1ff-umxt3-wg5qx-52ry2-qrpdk-ssz22-ws2kg-demj2-cd8zw-j2skz-x2wq0-qw70l-4q99v-dyh5x-0zxwq-65s8c-69ev9-z
md1ff-umxt3-4j5jr-7qyur-6vt76-esnw4-xmrk8-qe0yr-02mhq-rqxsh-ncevm-s69k3-57he6-px5mr-n2zz0-33uqk-qk8lq-l
md1ff-umxt3-u4308-vyy88-ae4he-w375v-hs43h-krycu-znj6y-tmt7h-5t2r3-u8dqp-82uxn-prxcu-ejudw-s6tu9-ntlfd-x
md1ff-umxtj-8xdwq-6zaqv-4n34y-v6nrr-6v8h9-q9d82-kwr34-7wwd9-mtxdy-pee03-z5f5x-vuvkk-vwu4e-plqpe-dw7ye-5
md1ff-umxtj-vfhzv-vd04v-33s0u-tarlj-6uxcp-hkqup-qc34f-lc5ln-hz7dz-kjmzw-fhtn2-tf9tp-83xfs-kawee-p093t-j
md1ff-umxtj-ks9c8-4gwag-yhhf8-ugmw8-t46ek-r0a7t-0c6gn-7uss8-mxrkma-a6e6m-kqp24-zvzxcn-clwk7-4rja8-2tmur-l
md1ff-umxtj-euccf-pazqp-0dmvslcy68-sh0lw-93nww-xth48-8eqjv-n67gq-jpx7f-yf84x-j7zjf-c2wr6-04yur-u2eq3-4
md1ff-umxtn-qams5-fd4v9-3wvkd-hx557-s4y6p-0jlmx-45mcl-995rj-zmgkc-6qdhh-aqgh6-68m2k-dsxy5-ru7p2-fk5qg-s
md1ff-umxtn-fmvzg-ueurv-mm8w8-gs4m5-lfqyq-yzsss-yy3fg-j9ee2-9zj26-l5afq-22n2c-t2h3z-wrj46-trnch-trjpt-z
md1ff-umxtn-sxr9y-3yqws-gv2re-7fhdv-psmee-edutt-gqw67-778e9-8hwxn-x0url-yv537-ntdqw-0kusp-3l3wy-yry2c-2
md1ff-umxtn-e55rt-tw8r3-a2s45-tsazm-rrfms-t4u6j-4atq6-uhd0m-3m5l9-lvu9c-de6pc-swdr7-3cph-9wc0u-tcf6l-5
md1ff-umxt5-x2zzh-awq5r-zqgrf-xdvhs-7fjqk-8nyx9-arf5p-wdpk-flzjs-yflyj-c6a90-76kng-ytumw-tunf5-hft8g-c
md1ff-umxt5-wtstc-3wnac-t9lau-qxytd-7f9xh-qmtml-ma4zh-svtx5-p6e3s-3rggp-2ezqz-979cu-ua2mf-vauyk-5ut07-p
md1ff-umxt5-nqn95-xg8r-20lnf-lydtq-clj74-3ynhl-29kg2-3r4kj-zs99q-yrk9a-808d5-f0gen-njsls-rzrrn-r484e-6
md1ff-umxt5-uc4jf-kr2yv-pq2d5-mclxj-xy0nu-qsfks-lrdsz-k03kw-csk2u-zz7sg-880kz-er6aw-9tk9y-pxcwz-cyw6f-y
md1ff-umxt4-zrtnj-mnh88-89ltv-hne3k-tx5e8-uhl2r-mxpaq-j6ey0-lvxj4-6th52-vv3c3-kju07-67gdj-m2nna-x3x5p-w
md1ff-umxt4-f8skf-ujv0x-falsr-xwl3g-9hkcc-vpd8x-sr264-u2ncg-unuyj-afvpj-wuctz-2cjex-lhznk-tnnzz-u3sml-7
md1ff-umxt4-30x4t-nf7n9-z8y7v-6ypn2-35vam-58em2-v583v-n4e8s-ye3c6-4t8u9-28ma9-zqcw3-udszu-lgfsx-u8xey-l
md1ff-umxt4-lluk3-chss0-cnthu-sns60-7jerr-dxzxu-h07l4-mhgs6-ut6px-jq3gs-642g7-8tg45

```
# === Wallet bundle: descriptor, mainnet ===
# Threshold: 3 of 11
# Cosigners:
# @0: (no ms1; watch-only),mk1:5a703 (73c5da0a @ m/84'/0'/0')
# @1: (no ms1; watch-only),mk1:5a702 (73c5da0a @ m/84'/0'/1')
# @2: (no ms1; watch-only),mk1:5a701 (73c5da0a @ m/84'/0'/2')
# @3: (no ms1; watch-only),mk1:5a700 (73c5da0a @ m/84'/0'/3')
# @4: (no ms1; watch-only),mk1:5a707 (b8688df1 @ m/84'/0'/0')
# @5: (no ms1; watch-only),mk1:5a706 (b8688df1 @ m/84'/0'/1')
# @6: (no ms1; watch-only),mk1:5a705 (b8688df1 @ m/84'/0'/2')
# @7: (no ms1; watch-only),mk1:5a704 (b8688df1 @ m/84'/0'/3')
# @8: (no ms1; watch-only),mk1:5a70b (28645006 @ m/84'/0'/0')
# @9: (no ms1; watch-only),mk1:5a70a (28645006 @ m/84'/0'/1')
# @10: (no ms1; watch-only),mk1:5a709 (28645006 @ m/84'/0'/2')
# Template: descriptor
# md1: 5a70
```

```
# Recovery: any 3 of 11 signing keys + md1 (template card).  
note: stdout is watch-only – public keys only, cannot spend
```

5.4 Watch-only export for Nunchuk / Core / Sparrow

There is no dedicated nunchuk emitter, but Nunchuk imports miniscript wallets from a **descriptor** or a **BSMS (BIP-129)** record -- both shown above in 5.2 (`--format descriptor` for *Add Wallet -> Import -> descriptor*, and `--format bsms` for the multisig import format, which also Bitcoin Core and Sparrow accept). After import, fund the address; each spend path opens only when its lock matures, and the hashlock tiers additionally require revealing the secret word's preimage X.

6. Taproot version of the degrading wallet

`wsh(...)` reveals the whole policy on every spend. **Taproot** gives a cooperative **key-path** spend (cheap, private, looks like single-sig) and splits the fallbacks into **script-tree leaves**, so a spend reveals only the leaf it uses. We keep the same four timelock/hash/multisig tiers as fallbacks and add a distinct cooperative internal key `Kint ([.../84'/0'/4'])`. Taproot multisig uses `multi_a`, not `multi`. **12 distinct keys** in total.

6.1 Why depth-1 (the one-tier-per-leaf limit on master)

The tidiest layout is one tier per leaf (4 leaves), but that is a **depth-2** taptree, and the shipped rust-miniscript pin mis-formats `depth->=2` taptrees (the PR-#953 bug). The toolkit refuses such a descriptor up front rather than emit a malformed one:

```
$ mnemonic export-wallet --descriptor "$(cat taproot-4leaf.desc)" --format descriptor --network mainnet
error: export-wallet script-type derive: taptree branch must have 2 children, but found 1
```

So we use a **depth-1** tree (2 leaves) and pack two tiers per leaf with `or_i`: Leaf A = tier 1 or tier 2 (the absolute-timelock + secret-word tiers); Leaf B = tier 3 or tier 4 (the relative-timelock tiers). (A rust-miniscript release > 13.1.0 containing #953 reopens deep trees -- tracked in FOLLOWUP taproot-coverage-cycle-on-miniscript-gt-13-1-0.)

6.2 Build + validate

Validate and canonicalise the hand-written `tr(...)` descriptor. The full descriptor, every xpub in full:

```
$ mnemonic export-wallet --descriptor "$(cat taproot.desc)" --format descriptor --network mainnet
tr([73c5da0a/84'/0'/4']xpub6CatWdiZiodmeXswr13Gd5aNtNqr2UHCBEsCol3eEFVaM7n8kY5kS4daaP83gWQncmzL3Wzt79mEiLiX6XZs6XQmGcQn
note: stdout is watch-only – public keys only, cannot spend
```

`Kint ([73c5da0a/84'/0'/4'])` is the key-path; the two `or_i(...)` blocks are the two script leaves; `after(...)` are the absolute (height/time) locks and `older(...)` the relative (blocks/time) locks -- the same four kinds as section 5.

6.3 Engrave + first address

Every key is distinct, so it engraves (watch-only). Take the `md1` chunks and restore to read the first address (this round-trip also proves the **real internal key** at the trunk reconstructs -- the v0.55.3 non-NUMS feature):

```
$ mnemonic bundle --descriptor-file taproot.desc --network mainnet --json > taproot.bundle.json
note: stdout is watch-only – public keys only, cannot spend
```

```
$ jq -r ".md1[]" taproot.bundle.json > taproot.md1
```

```
$ cat taproot.md1
md1f40gppvq9zem6jzwrh4yyym6jzwx75sj3m6jzt802ggrh4yyvaafp9rh4yykw75ss802ggemcn9wrf4m7m5yt9
md1f40gppvq2jz2sqrqgy2efntvqq7sjqfntk5ymnjqjatj0sucqg70h4gdtkemje3rw4fp6rc6cnnxqh4hnx7rv
md1f40gppvqscmmq5mngy26gzzz6v6mwrda3qzv6a4pxuusy2unu8xqz8naa2r2akwukvgm4g7w7kdyafR05mf
md1f40gppvq6gws7xkx9k7c9xu6pzkjqqj9vefntsqqpllllyqshsnxhqqgrcf5gqzn2cvasqu79cwukltfwjcnckn
md1f40gppvprg9pw0za5z3883w6pgmncdq53eutks2twrg3hckhp5gmutms6yd7x9cdzxlry5xgklam3a93x43x
md1f40gppvpg5qx52ry2qrt9pj9qpskufqq44hccjz40ltn8tdswlxxk6gs50wsfd6vu4jev3eggy8f0nxndxa6x
md1f40gppv56gl4vs3t2qswfth67hsept4g4hf5939sedkqhey4nnc0chj6t2x82xgqygmpkvg7cuw0m7akf723
md1f40gppv622ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0z8v4yslsp8q7nzlkkvymvslrcugrdkju72
md1f40gppvz9fkca3cxteqm6kacqcp59u7xtux3d5d847wsf4xev6setz7wcggw0mnt0jaragegqfesknkceccqj
md1f40gppvz0ptr0vxf3c9895ghkha0gk58rcw6qzw4cdxzd3en9mnxhqdpwsx2ec6jxdf33axqmwg8tzlv563mv
md1f40gppvznmjszkn4t8pc6l88xna4nxjqquh32y6rxwxttx8w3xuf33474jxxpl30507ttsmq84l460gcw0g4u
md1f40gppvzc7csyrzx48lzn7wute526tvfexawdfdy4vy7y7s9c84gwagyhhf8ugmwft46ekqd54ka5c8c48mx
md1f40gppvrr0a7t0c6gn7uss8mxrkma6e6mkqp24zvzxcnlw0xxzg0gsqtmwmy87px3u9ml3vgd8s0y6hvrurvud
md1f40gppvrvnm3jaafe7gyny7hjyqsfhjz4f5hs5jwzh3mhpjgm2ctzuevmwdfap2f5z19lsqqkg20rz8nj9q
md1f40gppvdkdth372tg8y9k3d35qm006q30450k4vmqvakywv7pkdanhr5g2a605sqzpgggqs9x9r4h0actx
md1f40gppv6zg55fzuu4z339d06w5s99f4v94tc38peqc5vjysp6pp3g08exa4srx0889h3ddqqnqhdl2vm2eucf
md1f40gppvpy6ttmcl57ac6vels0u3jj86dd5pe7m355rttw8r3a2s45tszcmrremst4u6j4atqccq4mtqrtm0ny
md1f40gppvpy6guhd0m3m5l9lvu9cde6pcswdr73ejss4lts9qcszq6fnt9u8jvs93uep30g6dqg0k3hj4xq9v7xn
md1f40gppvpyng0aj0c55pz0eykxhftlk456pzlvcqh3za8msktlmcqvkgmuj2dwpkhhlhm290qceu48zwwu535kp
md1f40gppvpyckdgr4nrpxssz4jyqytut3ee64sfnz6ryr348le5ljx4sv0e02cjfml4zmy9g36c6spmukzwf5w5f
```



```
md1f40gpv9rfpgzjjzpmz7nhnk6yh5veeegnzkfxcdg3sypfkn0ru6gc370szpx6rudkq2e7xecrr560mc7uy987
md1f40gpv9tzzetsgt6pqua7cty0t4c4w6rtnjmnht889ltvhne3ktx5etuhl2rmxpaqj6ey0lg6d4mp5l9hjrne
md1f40gpv95xj46th52vv3c3kju0767gfu9j0ynrej00uqenhu2pdakxrqtfe5qck40z57z8ylqssej5h2v929gq
md1f40gpv9eyh2tqvnhxzckz5kf hacj7d2lxnax2ywfue5grx4rgemhg0nk5eg0ze8tj0qfnr3syntujt2jgx5pl
md1f40gpvx92k0c250h62ypsarcml7tgutcg8uf4m7g fcd8l fv33knprwthl06am5gdw9aqnfqzkhq0nhyzweea
```

```
$ mnemonic restore --network mainnet $(sed "s/^/--md1 /" taproot.md1)
```

```
miniscript policy restore (12 cosigners)
```

```
CONFIRM: verify each cosigner fingerprint against your records before importing.
```

```
descriptor: tr([73c5da0a/84'/0'/4']xpub661MyMwAQbRcEyUKSqsBgaz1Lob8pCa1rM1SJS8CEzGCYyP9LisxZ2m1goDqj137XvHdY2nNkctqiE1i
```

```
first recv: bc1p9stcwz5597fmkxae9343k8edzkcvdczf9qp65r6p447pg0et82yqst3d2c
```

```
cosigner @0: 73c5da0a [84'/0'/4'] from md1 (not independently verified)
```

```
cosigner @1: 73c5da0a [84'/0'/0'] from md1 (not independently verified)
```

```
cosigner @2: 73c5da0a [84'/0'/1'] from md1 (not independently verified)
```

```
cosigner @3: 73c5da0a [84'/0'/2'] from md1 (not independently verified)
```

```
cosigner @4: 73c5da0a [84'/0'/3'] from md1 (not independently verified)
```

```
cosigner @5: b8688df1 [84'/0'/0'] from md1 (not independently verified)
```

```
cosigner @6: b8688df1 [84'/0'/1'] from md1 (not independently verified)
```

```
cosigner @7: b8688df1 [84'/0'/2'] from md1 (not independently verified)
```

```
cosigner @8: b8688df1 [84'/0'/3'] from md1 (not independently verified)
```

```
cosigner @9: 28645006 [84'/0'/0'] from md1 (not independently verified)
```

```
cosigner @10: 28645006 [84'/0'/1'] from md1 (not independently verified)
```

```
cosigner @11: 28645006 [84'/0'/2'] from md1 (not independently verified)
```

```
UNVERIFIED: no --from/--cosigner cross-check supplied; verify each cosigner fingerprint above against your records before importing
```

```
note: stdout is watch-only – public keys only, cannot spend
```

(Restore re-serialises each key as a depth-0 xpub661My... -- a different descriptor string, identical addresses; that is how the md1 card stores keys.)

6.4 Export for wallets (Nunchuk / Core / Sparrow)

descriptor and bitcoin-core both work for taproot:

```
$ mnemonic export-wallet --descriptor "$(cat taproot.desc)" --format bitcoin-core --network mainnet
```

```
[
  {
    "active": true,
    "desc": "tr([73c5da0a/84'/0'/4']xpub6CatWdiZiodmeXswr13Gd5aNtNqr2UHCBEsCoL3eEFVaM7n8kY5kS4daaP83gWQncmzL3Wzt79mEiLiXo",
    "internal": false,
    "range": [
      0,
      999
    ],
    "timestamp": 0
  },
  {
    "active": true,
    "desc": "tr([73c5da0a/84'/0'/4']xpub6CatWdiZiodmeXswr13Gd5aNtNqr2UHCBEsCoL3eEFVaM7n8kY5kS4daaP83gWQncmzL3Wzt79mEiLiXo",
    "internal": true,
    "range": [
      0,
      999
    ],
    "timestamp": 0
  }
]
```

```
note: stdout is watch-only – public keys only, cannot spend
```

But **BSMS / BIP-129** is not available for taproot (BIP-129 predates BIP-386) -- the toolkit says so and points you elsewhere:

```
$ mnemonic export-wallet --descriptor "$(cat taproot.desc)" --format bsms --network mainnet
error: --format bsms does not support taproot (P2trMulti); BIP-129 §1 prerequisites do not yet include BIP-386. Real emit support is tracked at FOLLOWUP `bsms-taproot-emit` and depends on a BIP-129 spec update. Use -format bitcoin-core (Core-importable) or --format sparrow (Sparrow JSON, taproot-capable) for taproot watch-only setup.
```

6.5 Adding a condition: which depth? (a spending-cost comparison)

Suppose you want a fifth spend path: a **new key Knew plus the preimage of a RIPEMD-160 hashlock** (secret word "please"). Should it be **folded** into an existing leaf (tree stays depth-1) or given its **own leaf** (forcing depth-2)? Decide on spending cost, not aesthetics. A taproot script-path witness costs: (satisfaction) + the **revealed leaf script** + a **control block** of 33 + 32*depth bytes. Folding many conditions into one leaf bloats *every* spend through it (you reveal the unused branches too); a deeper leaf adds one 32-byte hash per level. A witness byte weighs 1 WU = 0.25 vB, so **one extra depth level = +8 vB**.

The new hashlock (same two-step scheme): preimage X = sha256("please"), descriptor hash Hp = ripemd160(X):

```
$ python3 -c "import hashlib; X=hashlib.sha256(b'please').digest(); print('preimage X=', X.hex()); print('hash Hp  =', hashlib.ripemd160(X).hex())"
preimage X = 56ccc4dcfc96534b06fc0c08a301be24f13b491484d5d984953cc0dba9bbb89a
hash Hp    = 06d05e2f02fb90ddf98d8cd95d806ba12b27aff4
```

mnemonic compare-cost reports per-condition witness vbytes (key-agnostic -- abstract labels A,B,... are auto-dummy-keyed). **Folded** -- the new tier joins Leaf B, so spending it reveals all of tiers 3+4+5:

```
$ mnemonic compare-cost --miniscript "or_i(and_v(v:older(65535),multi(2,A,B)),or_i(and_v(v:older(4255898),multi(1,C,D,E)),and_v(v:pk(F),ripemd160(06d05e2f02fb90ddf98d8cd95d806ba12b27aff4)))"
Input: or_i(and_v(v:older(65535),multi(2,A,B)),or_i(and_v(v:older(4255898),multi(1,C,D,E)),and_v(v:pk(F),ripemd160(06d05e2f02fb90ddf98d8cd95d806ba12b27aff4)))
Wrapper comparison: wsh(M) vs tr(NUMS, {M})
Feerate: 1.0 sat/vB
```

Condition	wsh vB	tr vB	Δ vB	wsh sats	tr sats	Δ sats
-----+-----+-----+-----+-----+-----+-----						
A + B + older(blocks)	79	147	+68	79	147	+68
C + older(512s)	61	132	+71	61	132	+71
D + older(512s)	61	132	+71	61	132	+71
E + older(512s)	61	132	+71	61	132	+71
F + preimage(h0)	69	139	+70	69	139	+70

note: per-condition vbytes are rounded individually; absolute numbers may differ by ±1 from real-tx accounting, Δ values are correct

note: input contains hash-preimage fragments; preimage-known rows are enumerated assuming the user can supply each preimage no preimage knowledge is implied)

Dedicated -- the new tier is its own leaf, revealing only itself:

```
$ mnemonic compare-cost --miniscript "and_v(v:pk(F),ripemd160(06d05e2f02fb90ddf98d8cd95d806ba12b27aff4))"
Input: and_v(v:pk(F),ripemd160(06d05e2f02fb90ddf98d8cd95d806ba12b27aff4))
Wrapper comparison: wsh(M) vs tr(NUMS, {M})
Feerate: 1.0 sat/vB
```

Condition	wsh vB	tr vB	Δ vB	wsh sats	tr sats	Δ sats
-----+-----+-----+-----+-----+-----+-----						
F + preimage(h0)	68	90	+22	68	90	+22

note: per-condition vbytes are rounded individually; absolute numbers may differ by ±1 from real-tx accounting, Δ values are correct

note: input contains hash-preimage fragments; preimage-known rows are enumerated assuming the user can supply each preimage no preimage knowledge is implied)

compare-cost models each input as a single leaf at the tree root (depth-0, 33-byte control block); add the real Merkle depth (+8 vB per level):

Placement	revealed script	tr vB (depth-0)	real depth	+ control block	real tr
Folded into	whole	139	1 (still 2	+8	~147 vB
Leaf B	<code>or_i(t3,t4,t5)</code>		leaves)		
Dedicated leaf	just t5	90	2 (now 3	+16	~106 vB
			leaves)		

A dedicated leaf is ~41 vB (~28%) cheaper to spend the new condition: not revealing tiers 3+4 (~49 vB) beats the +8 vB for the extra depth level. It is better still in practice because folding also makes *t3* and *t4* spends reveal *t5*'s script, and a 3-leaf tree can place the **most-used** condition in the shallow slot and bury cold paths deeper.

Rule of thumb: in taproot, one-condition-per-leaf almost always wins -- each sibling script you avoid revealing is worth far more than the 8 vB/level of depth -- and you order leaves hot-shallow / cold-deep.

The catch: cost says depth-2, but master cannot build depth->=2 yet (the PR-#953 taptree-Display bug, 6.1), so today the only buildable option is the folded depth-1 leaf at the ~28% premium. That premium is the concrete, quantified motive for landing the upstream fix.

Appendix -- the public test seeds used

Slot	BIP-39 test phrase	Fingerprint (BIP-87)
@0	abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about	73c5da0a
@1	legal winner thank year wave sausage worth useful legal winner thank yellow	b8688df1
@2	letter advice cage absurd amount doctor acoustic avoid letter advice cage above	28645006

These are world-known BIP-39 vectors with no funds. Sections 5-6 derive their keys from these same three seeds at distinct `m/84'/0'/N'` accounts. Generated with `mnemonic v0.55.3` on Linux. See the in-repo manual (`docs/manual/`) for the authoritative per-flag reference.